UNIT-3

INFERENCE IN FIRST ORDER LOGIC

Inference in first order logic

- Inference Vs Propositional Logic
- Unification and Lifting
- Forward Chaining
- Backward Chaining
- Resolution

3.5 Inference Vs First Order Inference

- Modern logical inference starts with simple rules that eliminate quantifiers, producing quantifier-free sentences. This allows conversion of a first-order knowledge base into propositional logic, enabling the use of propositional inference.
- The next step introduces a shortcut: inference methods that work directly on first-order sentences.

3.5.1 Inference rules for quantifiers

- Suppose our knowledge base contains the standard folkloric axiom stating that all greedy kings are evil:

$$\forall x \; King(x) \wedge Greedy(x) \Rightarrow Evil(x)$$

Then it seems quite permissible to infer any of the following sentences:

$$King(John) \wedge Greedy(John) \Rightarrow Evil(John)$$

$$King(Richard) \wedge Greedy(Richard) \Rightarrow Evil(Richard)$$

$$King(Father\ (John)) \wedge Greedy(Father\ (John)) \Rightarrow Evil(Father\ (John))$$

Universal Instantiation

- The rule of Universal Instantiation (UI for short) says that we can infer any sentence obtained by substituting a ground term (a term without variables) for the variable.
- Let SUBST(θ,α) denote the result of applying the substitution θ to the sentence α. Then the rule is written

$$\frac{\forall v \ \alpha}{\text{SUBST}(\{v/g\}, \alpha)}$$

- for any variable v and ground term g. For example, the three sentences given earlier are obtained with the substitutions {x/John}, {x/Richard}, and {x/Father (John)}.

Existential Instantiation

- In the rule for Existential Instantiation, the variable is replaced by a single new constant symbol. The formal statement is as follows: for any sentence α, variable v, and constant symbol k that does not appear elsewhere in the knowledge base,

$$\frac{\exists v \ \alpha}{\text{SUBST}(\{v/k\}, \alpha)} .$$

For example, from the sentence

$$\exists \ x \ \text{Crown}(x) \land \text{OnHead}(x, \text{John})$$

we can infer the sentence

$$\text{Crown}(C_1) \land \text{OnHead}(C_1, \text{John})$$

Existential Instantiation

- As long as C_1 does not appear elsewhere in the knowledge base. Basically, the existential sentence says there is some object satisfying a condition, and applying the existential instantiation rule just gives a name to that object. Of course, that name must not already belong to another object.
- Mathematics provides a nice example: suppose we discover that there is a number that is a little bigger than 2.71828 and that satisfies the equation d(x^y)/ dy = x^y for x. We can give this number a name, such as e, but it would be a mistake to give it the name of an existing object, such as π.
- In logic, the new name is called a **Skolem constant**. Existential Instantiation is a special case of a more general process called **skolemization**,

Here's a **comparison table of Universal Instantiation (UI) vs Existential Instantiation (EI):**

Feature	Universal Instantiation (UI)	Existential Instantiation (EI)
Definition	Replace a variable in $\forall x\ \alpha$ with any ground term	Replace a variable in $\exists x\ \alpha$ with a **new constant** (Skolem constant)
Application	Can be applied **many times** with different substitutions	Applied **once**, then the existential sentence is discarded
Example	$\forall x$ [King(x) $\wedge$ Greedy(x) $\Rightarrow$ Evil(x)] $\rightarrow$ King(John) $\wedge$ Greedy(John) $\Rightarrow$ Evil(John)	$\exists x$ [Crown(x) $\wedge$ OnHead(x, John)] $\rightarrow$ Crown(C1) $\wedge$ OnHead(C1, John)
Constant used	Any existing ground term	A **fresh symbol** not appearing elsewhere (Skolem constant)
Logical effect	Produces multiple possible consequences	Gives a "name" to the unknown object
Equivalence	Maintains logical equivale	New KB not logically equivalent, but **inferentially equivalent**

3.5.2 Reduction to Propositional Inference

- The first idea is that, just as an existentially quantified sentence can be replaced by one instantiation, a universally quantified sentence can be replaced by the set of all possible instantiations. For example, suppose our knowledge base contains just the sentences

  $\forall$ x King(x) $\wedge$ Greedy(x) $\Rightarrow$ Evil(x)

  King(John)

  Greedy(John)

  Brother (Richard, John)

- Then we apply UI to the first sentence using all possible ground-term substitutions from the vocabulary of the knowledge base—in this case, {x/John} and {x/Richard}. We obtain
  - King(John) $\wedge$ Greedy(John) $\Rightarrow$ Evil(John)
  - King(Richard) $\wedge$ Greedy(Richard) $\Rightarrow$ Evil(Richard)
- and we discard the universally quantified sentence.
- Every first-order knowledge base and query can be propositionalized in such a way that entailment is preserved. Thus, we have a complete decision procedure for entailment ... or perhaps not. There is a problem: when the knowledge base includes a function symbol, the set of possible ground-term substitutions is infinite!

- For example, if the knowledge base mentions the Father symbol, then infinitely many nested terms such as Father (Father (Father (John))) can be constructed.
- Fortunately, there is a famous theorem due to Jacques Herbrand (1930) to the effect that if a sentence is entailed by the original, first-order knowledge base, then there is a proof involving just a finite subset of the propositionalized knowledge base.
- Since any such subset has a maximum depth of nesting among its ground terms, we can find the subset by first generating all the instantiations with constant symbols (Richard and John), then all terms of depth 1 (Father (Richard) and Father (John)), then all terms of depth 2, and so on, until we are able to construct a propositional proof of the entailed sentence.

3.6 Unification and Lifting

- A first-order inference rule
- Unification
- Storage and retrieval

The propositionalization approach (used before the 1960s) was inefficient because it generated unnecessary instances

$$\text{King}(\text{Richard}) \land \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$$

even when the query was only about John.

- For example, to infer Evil(John) from:
  - $\forall x(King(x) \land Greedy(x) \Rightarrow Evil(x))$
  - King(John),Greedy(John)
- A human sees the conclusion as obvious, and modern methods aim to make it just as obvious for a computer.
- Note: Old propositionalization was inefficient; direct inference makes results like Evil(John) straightforward.

3.6.1 A first-order inference rule

- The inference that John is evil—that is, that {x/John} solves the query Evil(x)—works like this: to use the rule that greedy kings are evil, find some x such that x is a king and x is greedy, and then infer that this x is evil.

- More generally, if there is some substitution θ that makes each of the conjuncts of the premise of the implication identical to sentences already in the knowledge base, then we can assert the conclusion of the implication, after applying θ.
- In this case, the substitution θ = {x/John} achieves that aim.
- Suppose that instead of knowing Greedy(John), we know that everyone is greedy:

  $\forall$ y Greedy(y) .
- Since John is included in "everyone," this means Greedy(John).
- So, combined with King(John), we can still infer Evil(John).

- In this case, applying the substitution {x/John, y/John} to the implication premises King(x) and Greedy(x) and the knowledge-base sentences King(John) and Greedy(y) will make them identical. Thus, we can infer the conclusion of the implication.
- This inference process can be captured as a single inference rule that we call Generalized Modus Ponens: For atomic sentences P_i, P_i', and q where there is a substitution θ such that SUBST(θ, pi) = SUBST(θ, pi), for all i,

$$\frac{p_1',\ p_2',\ \ldots,\ p_n',\ (p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q)}{\text{SUBST}(\theta, q)}$$

- There are n+1 premises to this rule: the n atomic sentences P_i' and the one implication. The conclusion is the result of applying the substitution θ to the consequent q. For our example:

p_1' is $King(John)$ $\quad\quad$ p_1 is $King(x)$

p_2' is $Greedy(y)$ $\quad\quad$ p_2 is $Greedy(x)$

θ is $\{x/John, y/John\}$ $\quad\quad$ q is $Evil(x)$

$\text{SUBST}(\theta, q)$ is $Evil(John)$.

First, we observe that, for any sentence p (whose variables are assumed to be universally quantified) and for any substitution θ,

p |= SUBST(θ, p)

holds by Universal Instantiation. It holds in particular for a θ that satisfies the conditions of the Generalized Modus Ponens rule. Thus, from $p_1', \ldots, p_n'$

we can infer

$$\text{SUBST}(\theta, p_1') \wedge \ldots \wedge \text{SUBST}(\theta, p_n')$$

and from the implication $p_1', \ldots, p_n'$ we can infer

$$\text{SUBST}(\theta, p_1) \wedge \ldots \wedge \text{SUBST}(\theta, p_n) \Rightarrow \text{SUBST}(\theta, q)$$

- Now, θ in Generalized Modus Ponens is defined so th $\text{SUBST}(\theta, p_i') = \text{SUBST}(\theta, p_i)$
- For all *i*; therefore the first of these two sentences matches the premise of the second exactly.
- Hence, SUBST(θ, q) follows by Modus Ponens.
- Generalized Modus Ponens is a lifted version of Modus Ponens. it raises Modus Ponens from ground (variable-free) propositional logic to first-order logic.

3.6.2 Unification

- Lifted inference rules require finding substitutions that make different logical expressions look identical. This process is called unification and is a key component of all first-order inference algorithms. The UNIFY algorithm takes two sentences and returns a unifier for them if one exists:

$$\text{UNIFY}(p, q) = \theta \text{ where } \text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$$

- finding all sentences in the knowledge base that unify with Knows(John, x). Here are the results of unification with four different sentences that might be in the knowledge base:

$$\begin{aligned}
&\text{UNIFY}(Knows(John, x),\ Knows(John, Jane)) = \{x/Jane\} \\
&\text{UNIFY}(Knows(John, x),\ Knows(y, Bill)) = \{x/Bill, y/John\} \\
&\text{UNIFY}(Knows(John, x),\ Knows(y, Mother(y))) = \{y/John, x/Mother(John)\} \\
&\text{UNIFY}(Knows(John, x),\ Knows(x, Elizabeth)) = fail\ .
\end{aligned}$$

- The last unification fails because x cannot take on the values John and Elizabeth at the same time. Now, remember that Knows(x, Elizabeth) means "Everyone knows Elizabeth,". so we should be able to infer that John knows Elizabeth. The problem arises only because the two sentences happen to use the same variable name, x.
- The problem can be avoided by standardizing apart one of the two sentences being unified, which means renaming its variables to avoid name clashes. For example, we can rename x in Knows(x, Elizabeth) to x_{17} (a new variable name) without changing its meaning. Now the unification will work:

$$\text{UNIFY}(Knows(John, x),\ Knows(x_{17}, Elizabeth)) = \{x/Elizabeth, x_{17}/John\}$$

3.6.3 Storage and retrieval

- Underlying the TELL and ASK functions used to inform and interrogate a knowledge base are the more primitive STORE and FETCH functions. STORE(s) stores a sentence s into the knowledge base and FETCH(q) returns all unifiers such that the query q unifies with some sentence in the knowledge base.
- The problem we used to illustrate unification—finding all facts that unify with Knows(John, x)—is an instance of FETCHing.
- The simplest way to implement STORE and FETCH is to keep all the facts in one long list and unify each query against every element of the list.

```
function UNIFY(x, y, θ) returns a substitution to make x and y identical
  inputs: x, a variable, constant, list, or compound expression
          y, a variable, constant, list, or compound expression
          θ, the substitution built up so far (optional, defaults to empty)

  if θ = failure then return failure
  else if x = y then return θ
  else if VARIABLE?(x) then return UNIFY-VAR(x, y, θ)
  else if VARIABLE?(y) then return UNIFY-VAR(y, x, θ)
  else if COMPOUND?(x) and COMPOUND?(y) then
      return UNIFY(x.ARGS, y.ARGS, UNIFY(x.OP, y.OP, θ))
  else if LIST?(x) and LIST?(y) then
      return UNIFY(x.REST, y.REST, UNIFY(x.FIRST, y.FIRST, θ))
  else return failure
```

```
function UNIFY-VAR(var, x, θ) returns a substitution

  if {var/val} ∈ θ then return UNIFY(val, x, θ)
  else if {x/val} ∈ θ then return UNIFY(var, val, θ)
  else if OCCUR-CHECK?(var, x) then return failure
  else return add {var/x} to θ
```

Figure 9.1 The unification algorithm. The algorithm works by comparing the structures of the inputs, element by element. The substitution θ that is the argument to UNIFY is built up along the way and is used to make sure that later comparisons are consistent with bindings that were established earlier. In a compound expression such as $F(A, B)$, the OP field picks out the function symbol F and the ARGS field picks out the argument list (A, B).

Query: Knows(John, x)

Sentence in KB: Brother(Richard, John)

- Knows and Brother are totally different function names. So, no matter what substitution we try, these two will never match. Trying to unify them is useless and inefficient.
- We can avoid such unifications by indexing the facts in the knowledge base.
- **predicate indexing:** puts all the Knows facts in one bucket and all the Brother facts in another. The buckets can be stored in a hash table for efficient access.
- Predicate indexing helps when there are many predicates but few clauses for each.
- But if a single predicate (like Employs(x, y)) has millions of employers and tens of millions of employees, the bucket becomes too large.

- Answering a query like Employs(x, Richard) would still require scanning the entire bucket, making it inefficient.

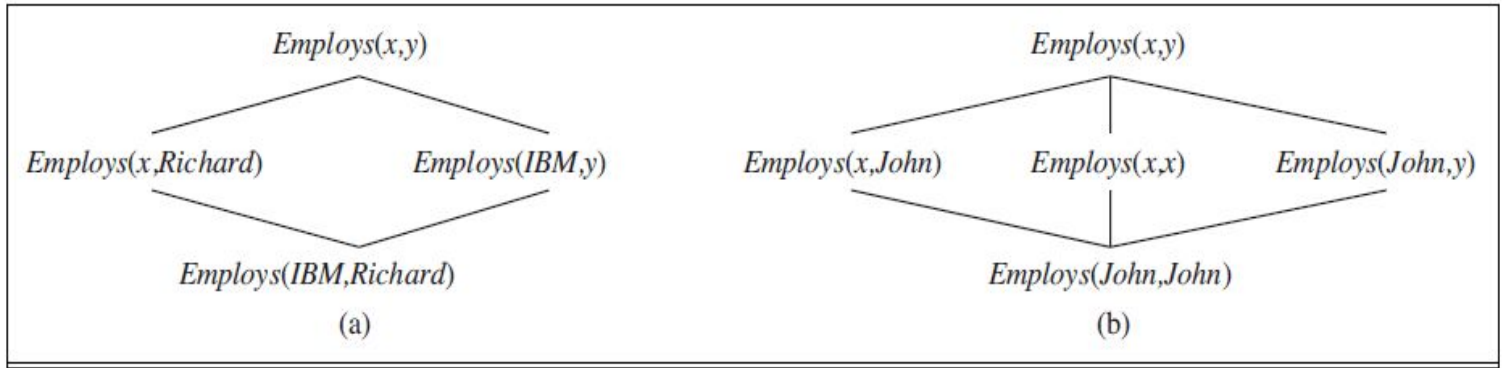


Figure 9.2 (a) The subsumption lattice whose lowest node is *Employs*(*IBM*, *Richard*). (b) The subsumption lattice for the sentence *Employs*(*John*, *John*).

- Predicate indexing alone is inefficient for large predicates like Employs(x, y) with millions of entries.
- Efficiency improves by indexing facts on predicate + argument(s) (e.g., by employer or by employee) using combined keys.

Example: Employs(IBM, Richard) supports queries like:

Employs(IBM, Richard) → Does IBM employ Richard?

Employs(x, Richard) → Who employs Richard?

Employs(IBM, y) → Whom does IBM employ?

Employs(x, y) → Who employs whom?

- These queries form a **subsumption lattice** where each child is derived by a substitution, and the most general unifier finds common descendants.
- Lattices are manageable for few nodes, but grow exponentially ($O(2^n)$) with arguments or function symbols.
- Too many indices become costly → so use fixed indexing policies (predicate + arguments) or adaptive indexing depending on queries.
- In AI systems (small data), indexing is solved; in commercial databases (billions of facts), indexing remains a major challenge.

3.7 Forward Chaining

- Forward Chaining start with the atomic sentences in the knowledge base and apply Modus Ponens in the forward direction, adding new atomic sentences, until no further inferences can be made.
- For first-order definite clauses (e.g., Situation $\Rightarrow$ Response), it's efficient and well-suited for systems that react to new information.

3.7.1 First-order definite clauses

- First-order definite clauses are like propositional ones: disjunctions with exactly one positive literal.
- They can be atomic (e.g., King(John)) or implications (e.g., King(x) $\wedge$ Greedy(x) $\Rightarrow$ Evil(x)).

$$King(x) \wedge Greedy(x) \Rightarrow Evil(x) .$$
$$King(John) .$$
$$Greedy(y) .$$

- Variables in them are universally quantified (assumed $\forall$). Not all knowledge bases can be written as definite clauses, but many can due to the single-positive-literal restriction.

Example:

The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by ColonelWest, who is American.

- We will prove that West is a criminal. First, we will represent these facts as first-order definite clauses. The next section shows how the forward-chaining algorithm solves the problem.

3.7.2 A simple forward-chaining algorithm

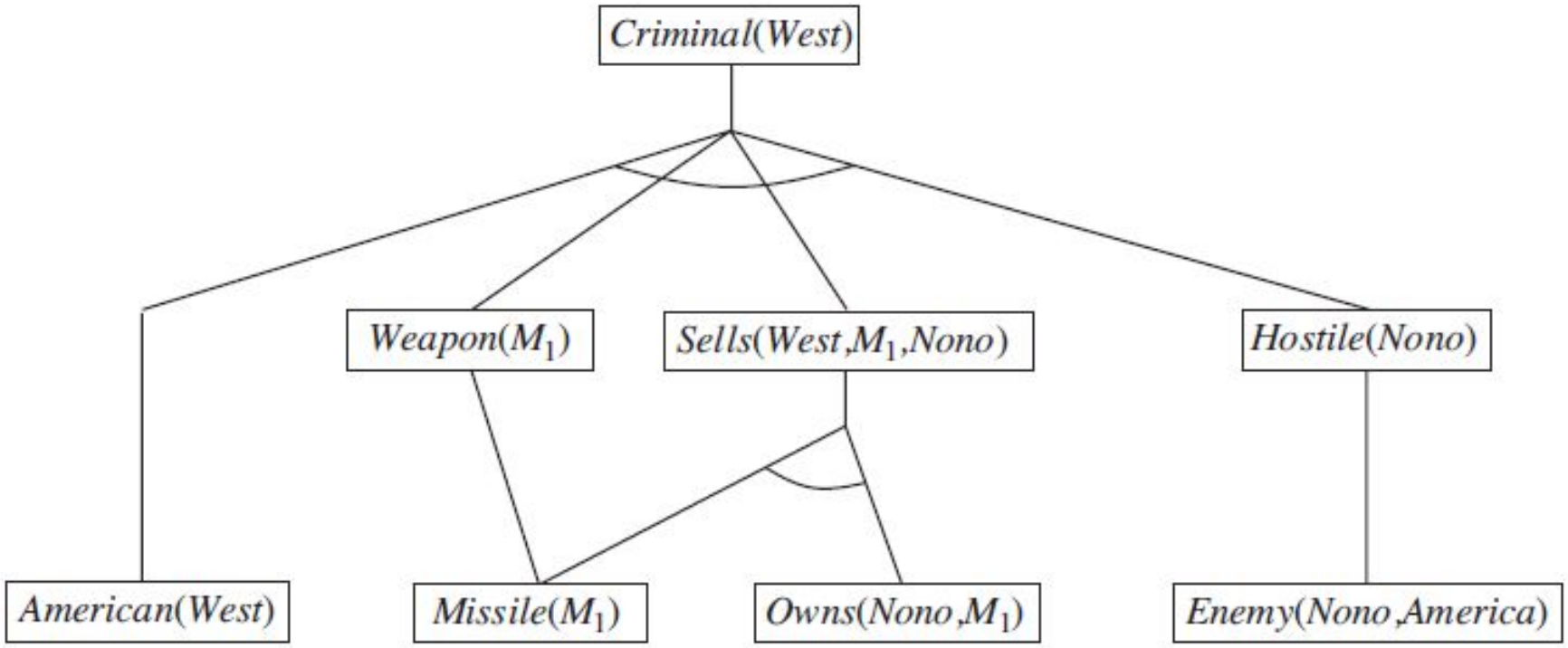


Figure 9.4 The proof tree generated by forward chaining on the crime example. The initial facts appear at the bottom level, facts inferred on the first iteration in the middle level, and facts inferred on the second iteration at the top level.

- It is a crime for an American to sell weapons to hostile nations":

  $American(x) \land Weapon(y) \land Sells(x, y, z) \land Hostile(z) \Rightarrow Criminal(x)$.

- "Nono . . . has some missiles." The sentence $\exists$ x Owns(Nono, x) $\land$Missile(x) is transformed into two definite clauses by Existential Instantiation, introducing a new constant M1:

  $Owns(Nono,M1)$

  $Missile(M1)$

- "All of its missiles were sold to it by Colonel West":

  $Missile(x) \land Owns(Nono, x) \Rightarrow Sells(West, x, Nono)$.

- We will also need to know that missiles are weapons:

  $Missile(x) \Rightarrow Weapon(x)$

- and we must know that an enemy of America counts as “hostile”:

  Enemy(x,America) ⇒ Hostile(x)

- “West, who is American . . .”:

  American(West) .

“The country Nono, an enemy of America . . .”:

  Enemy(Nono,America) .

- Datalog is a language of first-order definite clauses without function symbols.
- It’s named for its similarity to relational database statements.
- The absence of function symbols makes inference simpler and more efficient.

Knowledge base

Facts (initial):

- F1: $American(West)$
- F2: $Enemy(Nono, America)$
- F3: $Owns(Nono, M_1)$
- F4: $Missile(M_1)$

Rules:

- R1: $American(x) \land Weapon(y) \land Sells(x, y, z) \land Hostile(z) \Rightarrow Criminal(x)$
- R2: $Missile(m) \Rightarrow Weapon(m)$
- R3: $Owns(Nono, m) \land Missile(m) \Rightarrow Sells(West, m, Nono)$
- R4: $Enemy(u, America) \Rightarrow Hostile(u)$

Forward chaining (GMP applications)

Iteration 1

- From F4 with R2: unify $m \mapsto M_1$
  $\Rightarrow Weapon(M_1)$.
- From F3 & F4 with R3: unify $m \mapsto M_1$
  $\Rightarrow Sells(West, M_1, Nono)$.
- From F2 with R4: unify $u \mapsto Nono$
  $\Rightarrow Hostile(Nono)$.

Iteration 2

- Premises of R1 are now all satisfied:
  - $American(West)$ (F1)
  - $Weapon(M_1)$ (iter. 1)
  - $Sells(West, M_1, Nono)$ (iter. 1)
  - $Hostile(Nono)$ (iter. 1)
- Unifier $\theta = \{x \mapsto West,\ y \mapsto M_1,\ z \mapsto Nono\}$.
- Apply R1 with θ:
  $\Rightarrow Criminal(West)$.

No other new (non-renamed) facts arise, and the query is proved:

$$\boxed{Criminal(West)}$$

```
function FOL-FC-ASK(KB, α) returns a substitution or false
  inputs: KB, the knowledge base, a set of first-order definite clauses
          α, the query, an atomic sentence
  local variables: new, the new sentences inferred on each iteration

  repeat until new is empty
      new ← { }
      for each rule in KB do
          (p1 ∧ ... ∧ pn ⇒ q) ← STANDARDIZE-VARIABLES(rule)
          for each θ such that SUBST(θ, p1 ∧ ... ∧ pn) = SUBST(θ, p'1 ∧ ... ∧ p'n)
                        for some p'1, ..., p'n in KB
              q' ← SUBST(θ, q)
              if q' does not unify with some sentence already in KB or new then
                  add q' to new
                  φ ← UNIFY(q', α)
                  if φ is not fail then return φ
      add new to KB
  return false
```

Figure 9.3 A conceptually straightforward, but very inefficient, forward-chaining algorithm. On each iteration, it adds to KB all the atomic sentences that can be inferred in one step from the implication sentences and the atomic sentences already in KB. The function STANDARDIZE-VARIABLES replaces all variables in its arguments with new ones that have not been used before.

- Let k be the maximum arity (number of arguments) of any predicate, p be the number of predicates, and n be the number of constant symbols.
- For general definite clauses with function symbols, FOL-FC-ASK can generate infinitely many new facts, so we need to be more careful. For the case in which an answer to the query sentence q is entailed, we must appeal to Herbrand's theorem to establish that the algorithm will find a proof.
- If the query has no answer, the algorithm could fail to terminate in some cases. For example, if the knowledge base includes the Peano axioms

  NatNum(0)

  $\forall$ n NatNum(n) $\Rightarrow$ NatNum(S(n)) ,

- then forward chaining adds NatNum(S(0)), NatNum(S(S(0))), NatNum(S(S(S(0)))), and so on.

3.7.3 Efficient Forward Chaining:

- The forward-chaining algorithm is designed for ease of understanding rather than for efficiency of operation.
- First, the "inner loop" of the algorithm involves finding all possible unifiers such that the premise of a rule unifies with a suitable set of facts in the knowledge base. This is often called pattern matching and can be very expensive.
- Second, the algorithm rechecks every rule on every iteration to see whether its premises are satisfied, even if very few additions are made to the knowledge base on each iteration.
- Finally, the algorithm might generate many facts that are irrelevant to the goal. We address each of these issues in turn.

- The conjunct ordering problem arises when matching rule premises (with multiple conditions) against facts in the knowledge base. The goal is to decide in which order to test the conjuncts to minimize cost. For example, in

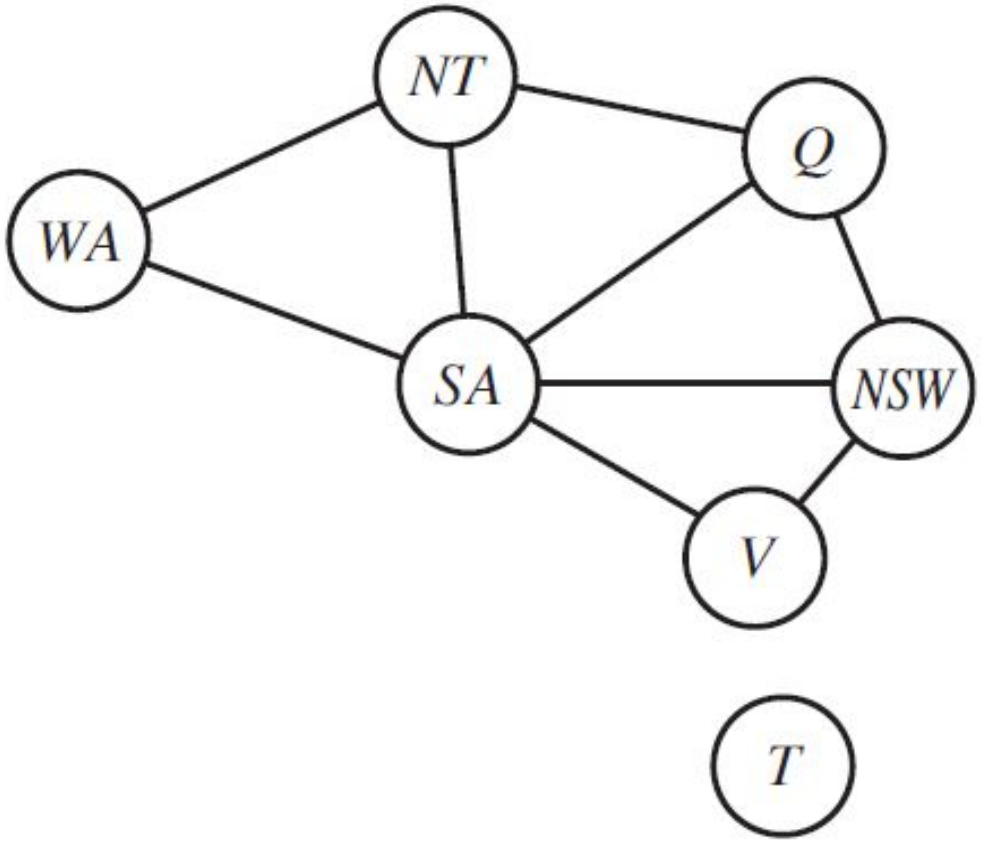
$$\text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$$

we can either:

- Find all objects Nono owns, then check which are missiles, or
- Find all missiles, then check which ones Nono owns.

The second is cheaper if missiles are fewer.

Finding the optimal ordering is NP-hard, but heuristics like MRV (minimum-remaining-values) help by testing the most restrictive conjunct first.



(a)

$$Diff(wa, nt) \wedge Diff(wa, sa) \wedge$$
$$Diff(nt, q) \wedge Diff(nt, sa) \wedge$$
$$Diff(q, nsw) \wedge Diff(q, sa) \wedge$$
$$Diff(nsw, v) \wedge Diff(nsw, sa) \wedge$$
$$Diff(v, sa) \Rightarrow Colorable()$$

$$Diff(Red, Blue) \quad Diff(Red, Green)$$
$$Diff(Green, Red) \quad Diff(Green, Blue)$$
$$Diff(Blue, Red) \quad Diff(Blue, Green)$$

(b)

Figure 9.5 (a) Constraint graph for coloring the map of Australia. (b) The map-coloring CSP expressed as a single definite clause. Each map region is represented as a variable whose value can be one of the constants *Red*, *Green* or *Blue*.

- Pattern matching in forward chaining is closely related to CSPs: each conjunct is like a constraint. Since CSPs (and 3-SAT) are NP-hard, rule matching is also NP-hard in general.
- But it's manageable because:
  - Data complexity is polynomial if rule size and predicate arity are small.
  - Tractable subclasses exist (e.g., tree-structured CSPs → linear-time matching).
  - Redundancy elimination in rule matching can speed things up.
- So, despite worst-case hardness, forward chaining is practical in real-world KBs.

- (a) Constraint graph for coloring the map of Australia
  - Each node = a region (WA, NT, Q, NSW, V, SA, T).
  - Each edge = "must have different colors" constraint between adjacent regions.
  - Example: WA is adjacent to NT and SA → so WA ≠ NT and WA ≠ SA.
- This is the graph representation of the CSP (Constraint Satisfaction Problem).
- (b) CSP expressed as a single definite clause
- All adjacency constraints are written as Diff(x, y), meaning "region x and region y must have different colors." The big clause is:
- $\text{Diff(wa,nt)} \land \text{Diff(wa,sa)} \land \text{Diff(nt,q)} \land \text{Diff(nt,sa)} \land \text{Diff(q,nsw)} \land \text{Diff(q,sa)} \land \text{Diff(nsw,v)} \land \text{Diff(nsw,sa)} \land \text{Diff(v,sa)} \Rightarrow \text{Colorable()}$

- This means: if all neighboring pairs of regions are colored differently, then the map is colorable.
- Ground facts (the allowed color differences)
- Below the clause you see things like:
  - Diff(Red, Blue)
  - Diff(Red, Green)
  - Diff(Green, Blue)etc.
- These are the basic facts saying which color pairs are different. Together, they define what "Diff" actually means.
- So, if wa = Red, then to satisfy Diff(wa, nt), we must have nt = Blue or nt = Green.

- How this works
  - Assign colors (Red, Green, Blue) to each region (variables).
  - Use the Diff facts to enforce constraints.
  - If all Diff constraints are satisfied, forward chaining can infer Colorable().

Incremental forward chaining

- Forward chaining can waste effort by repeatedly re-matching rules with already known facts.

Improvements:

- Incremental forward chaining: At iteration t, only check rules involving facts newly inferred at t–1.
- Partial match storage: Keep unfinished matches instead of rebuilding them each time.
- Rate algorithm: Builds a network of rule premises, keeps partial matches, and filters variable bindings efficiently.
- Used in production systems (e.g., XCON, OPS-5) and cognitive architectures (ACT, SOAR), enabling real-time reasoning with millions of rules.

3.8 Backward Chaining

- Backward chaining is a logical inference method that starts from a goal (query) and works backward to determine if it can be supported by known facts and rules. It is widely used in logic programming (e.g., Prolog).
- The method is efficient for answering specific queries but can face disadvantages such as redundant searches or infinite loops.
- To address this, optimizations and connections with constraint satisfaction problems are explored.

3.8.1 A backward-chaining algorithm

```
function FOL-BC-ASK(KB, query) returns a generator of substitutions
  return FOL-BC-OR(KB, query, { })

generator FOL-BC-OR(KB, goal, θ) yields a substitution
  for each rule (lhs ⇒ rhs) in FETCH-RULES-FOR-GOAL(KB, goal) do
    (lhs, rhs) ← STANDARDIZE-VARIABLES((lhs, rhs))
    for each θ′ in FOL-BC-AND(KB, lhs, UNIFY(rhs, goal, θ)) do
      yield θ′

generator FOL-BC-AND(KB, goals, θ) yields a substitution
  if θ = failure then return
  else if LENGTH(goals) = 0 then yield θ
  else do
    first, rest ← FIRST(goals), REST(goals)
    for each θ′ in FOL-BC-OR(KB, SUBST(θ, first), θ) do
      for each θ″ in FOL-BC-AND(KB, rest, θ′) do
        yield θ″
```

Figure 9.6 A simple backward-chaining algorithm for first-order knowledge bases.

A backward-chaining algorithm

- FOL-BC-ASK(KB, goal) will be proved if the knowledge base contains a clause of the form lhs $\Rightarrow$ goal, where lhs (left-hand side) is a list of conjuncts. An atomic fact like American(West) is considered as a clause whose lhs is the empty list. Now a query that contains variables might be proved in multiple ways.
- For example, the query Person(x) could be proved with the substitution {x/John} as well as with {x/Richard }. So we implement FOL-BC-ASK as a function that returns multiple times, each time giving one possible result.
- Backward chaining is a kind of AND/OR search—the OR part because the goal query can be proved by any rule in the knowledge base, and the AND part because all the conjuncts in the lhs of a clause must be proved.

- FOL-BC-OR works by fetching all clauses that might unify with the goal, standardizing the variables in the clause to be brand-new variables, and then, if the rhs of the clause does indeed unify with the goal, proving every conjunct in the lhs, using FOL-BC-AND.
- Backward chaining, as we have written it, is clearly a depth-first search algorithm. This means that its space requirements are linear in the size of the proof (neglecting, for now, the space required to accumulate the solutions).
- It also means that backward chaining (unlike forward chaining) suffers from problems with repeated states and incompleteness.

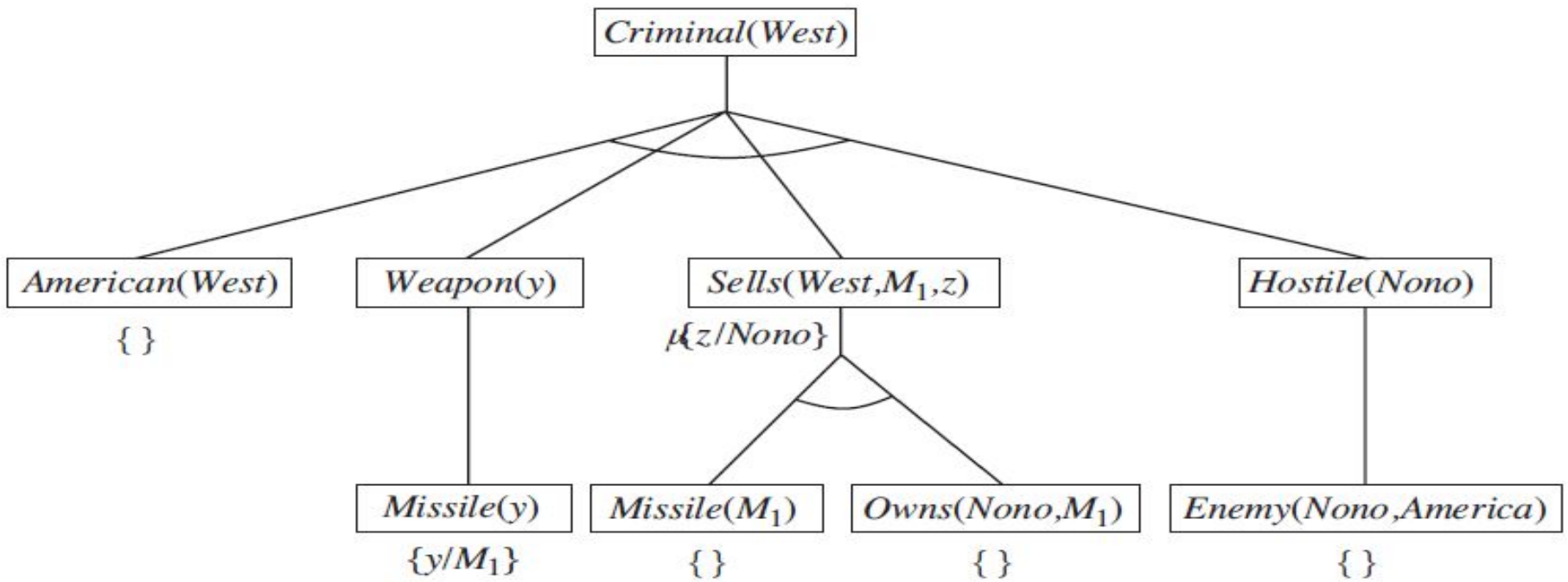


Figure 9.7 Proof tree constructed by backward chaining to prove that West is a criminal. The tree should be read depth first, left to right. To prove *Criminal*(*West*), we have to prove the four conjuncts below it. Some of these are in the knowledge base, and others require further backward chaining. Bindings for each successful unification are shown next to the corresponding subgoal. Note that once one subgoal in a conjunction succeeds, its substitution is applied to subsequent subgoals. Thus, by the time FOL-BC-ASK gets to the last conjunct, originally *Hostile*(*z*), *z* is already bound to *Nono*.

Explanations of the notations on the tree

- Each node is a goal/subgoal.
- Braces like {} next to a leaf mean “a fact was used, no new substitution.”
- A brace like { y / M1 } next to a node means that node produced the binding y = M1 by unification.
- μ{z/Nono} near the Sells node in the figure indicates that proving Sells produced the binding z = Nono (often called μ or θ for the substitution). That substitution is then applied to subsequent subgoals in the conjunction.

- Goal

  Criminal(West)

- We try to prove this by backward chaining using rule (1). Unify the goal with the head of rule (1):

  Unify Criminal(West) with Criminal(x) ⟶ substitution θ = { x / West }.

- So we must prove the body of (1) with x replaced by West:

  American(West) ∧ Weapon(y) ∧ Sells(West, y, z) ∧ Hostile(z)

- The tree in the figure shows these four conjuncts as four children of the root.

- Step-by-step proof (depth-first, left-to-right)
- 1. Prove American(West)
  - This is a fact in the KB (American(West)), so it succeeds immediately.
  - No new substitution: {}.
  - Cumulative substitution so far: { x / West } (from the top).
- 2. Prove Weapon(y)
  - Use rule Weapon(y) ← Missile(y).
  - Need to prove Missile(y).
  - Missile(M1) is a fact.
  - Unify Missile(y) with Missile(M1) → substitution θ1 = { y / M1 }.
  - So Weapon(y) succeeds with y bound to M1.
- Cumulative substitution now: { x/West, y/M1 }.

- Prove Sells(West, y, z) with current y = M1
  - After applying y/M1, this subgoal becomes Sells(West, M1, z).
  - Use the rule Sells(x,y,z) ← Missile(y) ∧ Owns(z,y).
  - Unify Sells(West,M1,z) with Sells(x,y,z) → substitution (consistent with earlier ones) gives { x/West, y/M1 } (these match existing bindings).
  - Now we must prove the body: Missile(M1) and Owns(z, M1).
    - Missile(M1) is a fact → succeeds ({}).
    - Owns(z, M1) must match Owns(Nono, M1) → unify gives θ2 = { z / Nono }.
  - So Sells(West,M1,z) succeeds and yields z bound to Nono.
- Cumulative substitution now: { x/West, y/M1, z/Nono }.

- 4. Prove Hostile(z) — but remember z is now bound
  - Because z was bound by the successful proof of Sells(...), the remaining conjunct Hostile(z) is now Hostile(Nono).
  - Use Hostile(z) ← Enemy(z, America). Prove Enemy(Nono, America).
  - Enemy(Nono, America) is a fact → succeeds.
- So Hostile(z) succeeds under z = Nono.
- **Result:**
- All four conjuncts succeed under the final composed substitution:

  $\mu = \{ x / West, y / M1, z / Nono \}$.
- Thus Criminal(West) is proven. The figure's root is true with that substitution.

3.8.2 Logic Programming

- The ideal is summed up in Robert Kowalski's equation,
- Algorithm = Logic + Control

Prolog Overview

- Logic Programming Language (most widely used).
- **Applications:** rapid prototyping, symbol manipulation (compilers, NLP), expert systems (legal, medical, financial).

Notation:

- **Variables:** Uppercase (e.g., X)
- **Constants:** Lowercase (e.g., john)
- **Clause form:** Head :- Body.
- **Equivalent to:** $A \wedge B \Rightarrow C \leftrightarrow C :- A, B$

Example Rule

criminal(X) :- american(X), weapon(Y), sells(X,Y,Z), hostile(Z).

Means: X is a criminal if X is American, owns a weapon Y, sells it to Z, and Z is hostile.

Lists in Prolog

Notation: [E|L] → list with head E and tail L.

Append Program

append([],Y,Y)

append([A|X],Y,[A|Z]) :- append(X,Y,Z).

Query Example ?- append(X,Y,[1,2]).

$(X,Y) \in \{([],[1,2]),([1],[2]),([1,2],[])\}$

Execution

Search strategy: Depth-first backward chaining.

Special cases:

- Arithmetic: "X is 4+3" → X=7
- "5 is X+Y" → fails
- Built-in predicates: assert/1, retract/1, I/O.
- Occur check omitted → possible unsound inference.
- No infinite recursion check → may loop forever.

3.8.3 Efficient implementation of logic programs

Prolog Execution Modes

Interpreted Mode

- Works like FOL-BC-ASK (backward chaining).
- Uses choice points stack (global structure) instead of manual iteration.
- Uses logic variables instead of explicit substitutions.
  - Each variable = either unbound or bound.
  - Trail stack keeps track of bindings for backtracking.
- On failure → backtrack to choice point → unbind variables from trail.
- Drawback: Each inference step = several thousand machine instructions (slow, because it must always re-match clauses).

Compiled Mode

- Program is translated to machine code (or specialized virtual instructions).
- Avoids repeated clause lookup and unification overhead.
- Runs much faster than interpreted mode.
- Behaves as if interpreter has “seen the program before.”

Key Idea

- Interpreted Prolog → flexible, debugging-friendly, but slower.
- Compiled Prolog → optimized execution, fewer repeated lookups, much faster.

```
procedure APPEND(ax, y, az, continuation)

    trail ← GLOBAL-TRAIL-POINTER()
    if ax = [ ] and UNIFY(y, az) then CALL(continuation)
    RESET-TRAIL(trail)
    a, x, z ← NEW-VARIABLE(), NEW-VARIABLE(), NEW-VARIABLE()
    if UNIFY(ax, [a | x]) and UNIFY(az, [a | z]) then APPEND(x, y, z, continuation)
```

Figure 9.8 Pseudocode representing the result of compiling the `Append` predicate. The function NEW-VARIABLE returns a new variable, distinct from all other variables used so far. The procedure CALL(*continuation*) continues execution with the specified continuation.

Key ideas:

- Continuations are used to explore multiple solutions (choice points).
- Parallelism can speed execution further:
  - OR-parallelism: explore multiple matching clauses in parallel.
  - AND-parallelism: solve conjuncts in parallel (harder due to shared variables).

3.8.4 Redundant inference and infinite loops

Prolog's weakness comes from its depth-first search:

Problem 1 – Infinite paths:

The order of rules can cause Prolog to follow an infinite recursion (e.g., path-finding example). This makes it incomplete—it may miss valid solutions.

Problem 2 – Redundant computations: Prolog repeatedly recomputes the same subgoals, leading to exponential work. For graphs, this is the repeated-state problem.

Forward chaining avoids these issues by caching results (dynamic programming), ensuring at most n^2 facts for n nodes.

Fix in backward chaining:

Use memoization (tabled logic programming) → cache and reuse subgoal results.

- Keeps backward chaining's goal-directed search.
- Gains forward chaining's efficiency.
- Makes Prolog complete for Datalog (no missed answers due to infinite loops).

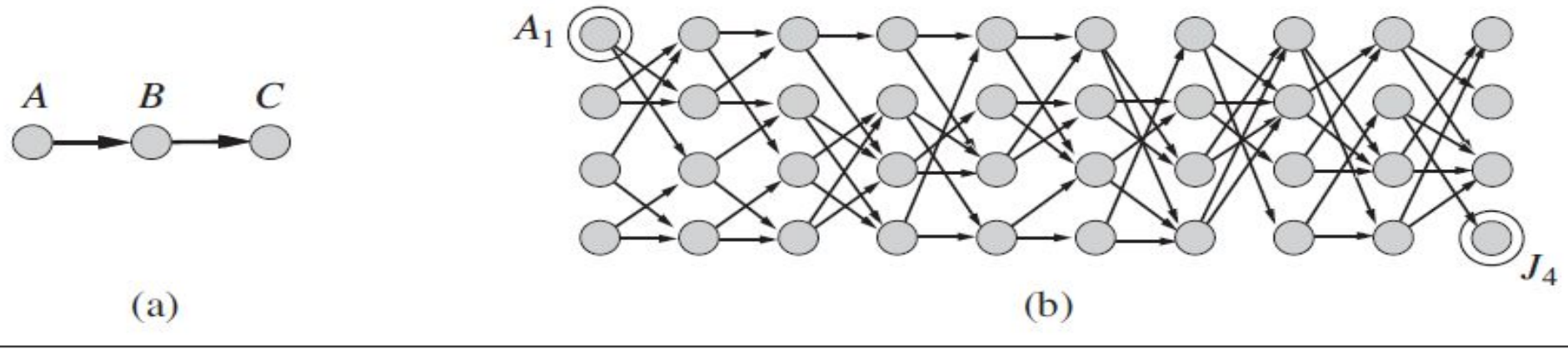


Figure 9.9 (a) Finding a path from A to C can lead Prolog into an infinite loop. (b) A graph in which each node is connected to two random successors in the next layer. Finding a path from A_1 to J_4 requires 877 inferences.

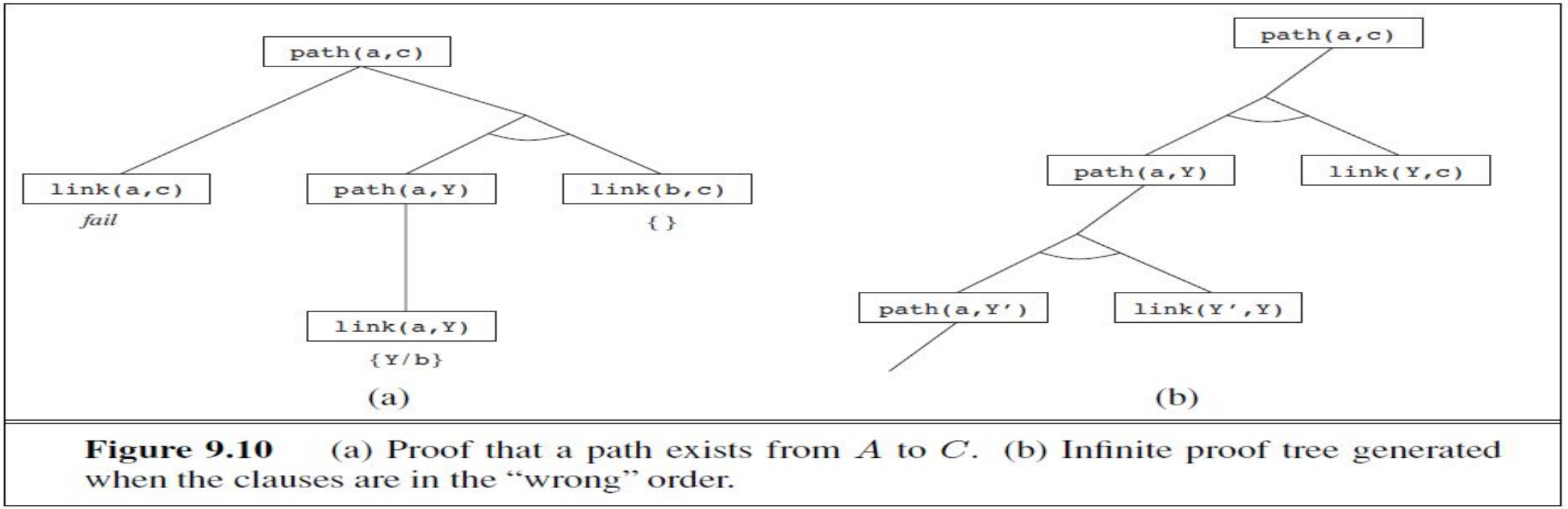


Figure 9.10 (a) Proof that a path exists from A to C. (b) Infinite proof tree generated when the clauses are in the "wrong" order.

3.8.5 Database semantics of Prolog

Prolog follows database semantics:

- Unique Names Assumption (UNA): Each constant/ground term refers to a different object (e.g., CS ≠ EE).
- Closed World Assumption (CWA): Only facts in the knowledge base are true; anything not listed is false.

Example:

Course(CS,101), Course(CS,102), Course(CS,106), Course(EE,101).

→ Exactly four distinct courses (no more, no less).

In FOL, the same assertions would mean:

- There are at least 1 course, maybe infinitely many.
- No guarantee that CS and EE or the numbers are distinct.

To capture Prolog's meaning in FOL, we need completion axioms (explicitly listing all true cases and enforcing equality distinctions).

Standard Prolog solves CSPs using backtracking search, which only works for finite domains.

Example: diff(Q,SA) with 3 colors → 6 solutions.

But queries with unbound variables in infinite domains (e.g., real numbers) fail.

3.8.6 Constraint logic programming

- Extends Prolog by allowing constraints (like inequalities) instead of only bindings
- Consider the following example. We define triangle(X,Y,Z) as a predicate that

holds if the three arguments are numbers that satisfy the triangle inequality:

Example: triangle(X,Y,Z) :-

X>0, Y>0, Z>0, X+Y>=Z, Y+Z>=X, X+Z>=Y.

If we ask Prolog the query triangle(3,4,5), it succeeds. On the other hand, if we ask triangle(3,4,Z), no solution will be found, because the subgoal Z>=0 cannot be handled by Prolog; we can't compare an unbound value to 0.

Advantages of CLP:

- Handles infinite-domain CSPs.
- Uses smarter search (heuristics, backjumping, cutset conditioning, etc.) instead of plain backtracking.
- Combines ideas from CSPs, logic programming, and deductive databases.

Advanced control: Some systems (e.g., MRS) let programmers write metarules to control search order, such as trying goals with fewer variables first.

Prolog = backtracking for finite CSPs; CLP = Prolog + constraints + advanced CSP algorithms, making it far more powerful.

3.9 Resolution

3.9.1 Conjunctive normal form for first-order logic

- As in the propositional case, first-order resolution requires that sentences be in conjunctive normal form (CNF)—that is, a conjunction of clauses, where each clause is a disjunction of literals.
- Literals can contain variables, which are assumed to be universally quantified. For example, the sentence

$$\forall x \ American(x) \land Weapon(y) \land Sells(x, y, z) \land Hostile(z) \Rightarrow Criminal(x)$$

becomes, in CNF

$$\neg American(x) \lor \neg Weapon(y) \lor \neg Sells(x, y, z) \lor \neg Hostile(z) \lor Criminal(x)$$

- Every sentence of first-order logic can be converted into an inferentially equivalent CNF sentence.
- "Everyone who loves all animals is loved by someone," or

$$\forall x [\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)] \Rightarrow [\exists y \text{ Loves}(y, x)] .$$

- The steps are as follows:

Eliminate implications:

$$\forall x [\neg \forall y \neg \text{Animal}(y) \vee \text{Loves}(x, y)] \vee [\exists y \text{ Loves}(y, x)] .$$

Move ¬ inwards: In addition to the usual rules for negated connectives, we need rules for negated quantifiers. Thus, we have

$\neg \forall x\ p$ becomes $\exists x \neg p$

$\neg \exists x\ p$ becomes $\forall x \neg p$.

- Skolemize: Skolemization is the process of removing existential quantifiers by elimination.
- Translate $\exists x\ P(x)$ into P(A), where A is a new constant. However, we can't apply Existential Instantiation to our sentence above because it doesn't match the pattern $\exists v\ \alpha$;
- only parts of the sentence match the pattern. If we blindly apply the rule to the two matching parts we get

$$\forall x\ [\text{Animal}(A) \wedge \neg \text{Loves}(x,A)] \vee \text{Loves}(B, x)\ ,$$

- which has the wrong meaning entirely: it says that everyone either fails to love a particular animal A or is loved by some particular entity B. In fact, our original sentence allows each person to fail to love a different animal or to be loved by a different person.
- Thus, we want the Skolem entities to depend on x and z:

$\forall$ x [Animal (F(x)) $\wedge$ ¬Loves(x, F(x))] $\vee$ Loves(G(z), x) .

- Here F and G are Skolem functions. The general rule is that the arguments of the Skolem function are all the universally quantified variables in whose scope the existential quantifier appears. As with Existential Instantiation, the Skolemized sentence is satisfiable exactly when the original sentence is satisfiable.
- **Drop universal quantifiers:** At this point, all remaining variables must be universally quantified. Moreover, the sentence is equivalent to one in which all the universal quantifiers have been moved to the left. We can therefore drop the universal quantifiers:

[Animal (F(x)) $\wedge$ ¬Loves(x, F(x))] $\vee$ Loves(G(z), x) .

Distribute $\vee$ over $\wedge$:

[Animal (F(x)) $\vee$ Loves(G(z), x)] $\wedge$ [¬Loves(x, F(x)) $\vee$ Loves(G(z), x)] .

This step may also require flattening out nested conjunctions and disjunctions

3.9.2 The resolution inference rule

Two clauses, which are assumed to be standardized apart so that they share no variables, can be resolved if they contain complementary literals. Propositional literals are complementary if one is the negation of the other; first-order literals are complementary if one unifies with the negation of the other. Thus, we have

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \qquad m_1 \vee \cdots \vee m_n}{\text{SUBST}(\theta, \ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n)}$$

where $\text{UNIFY}(\ell_i, \neg m_j) = \theta$. For example, we can resolve the two clauses

$$[Animal(F(x)) \vee Loves(G(x), x)] \quad \text{and} \quad [\neg Loves(u, v) \vee \neg Kills(u, v)]$$

- by eliminating the complementary literals Loves(G(x), x) and ¬Loves(u, v), with unifier

  θ={u/G(x), v/x}, to produce the resolvent clause

  [Animal (F(x)) ∨ ¬Kills(G(x), x)] .

- This rule is called the binary resolution rule because it resolves exactly two literals.

- Binary resolution alone is incomplete. Full resolution allows resolving any unifiable literals in clauses. Factoring removes redundant literals—identical ones in propositional logic, unifiable ones in first-order logic (with the unifier applied to the whole clause). Together, binary resolution and factoring form a complete inference procedure.

3.9.3 Example proofs

- Resolution proves that KB |= α by proving KB ∧ ¬α unsatisfiable, that is, by deriving the empty clause.
- The sentences in CNF are

  ¬American(x) ∨ ¬Weapon(y) ∨ ¬Sells(x, y, z) ∨ ¬Hostile(z) ∨ Criminal (x)

  ¬Missile(x) ∨ ¬Owns(Nono, x) ∨ Sells(West, x, Nono)

  ¬Enemy(x,America) ∨ Hostile(x)

  ¬Missile(x) ∨Weapon(x)

  Owns(Nono,M1) Missile(M1)

  American(West) Enemy(Nono,America) .

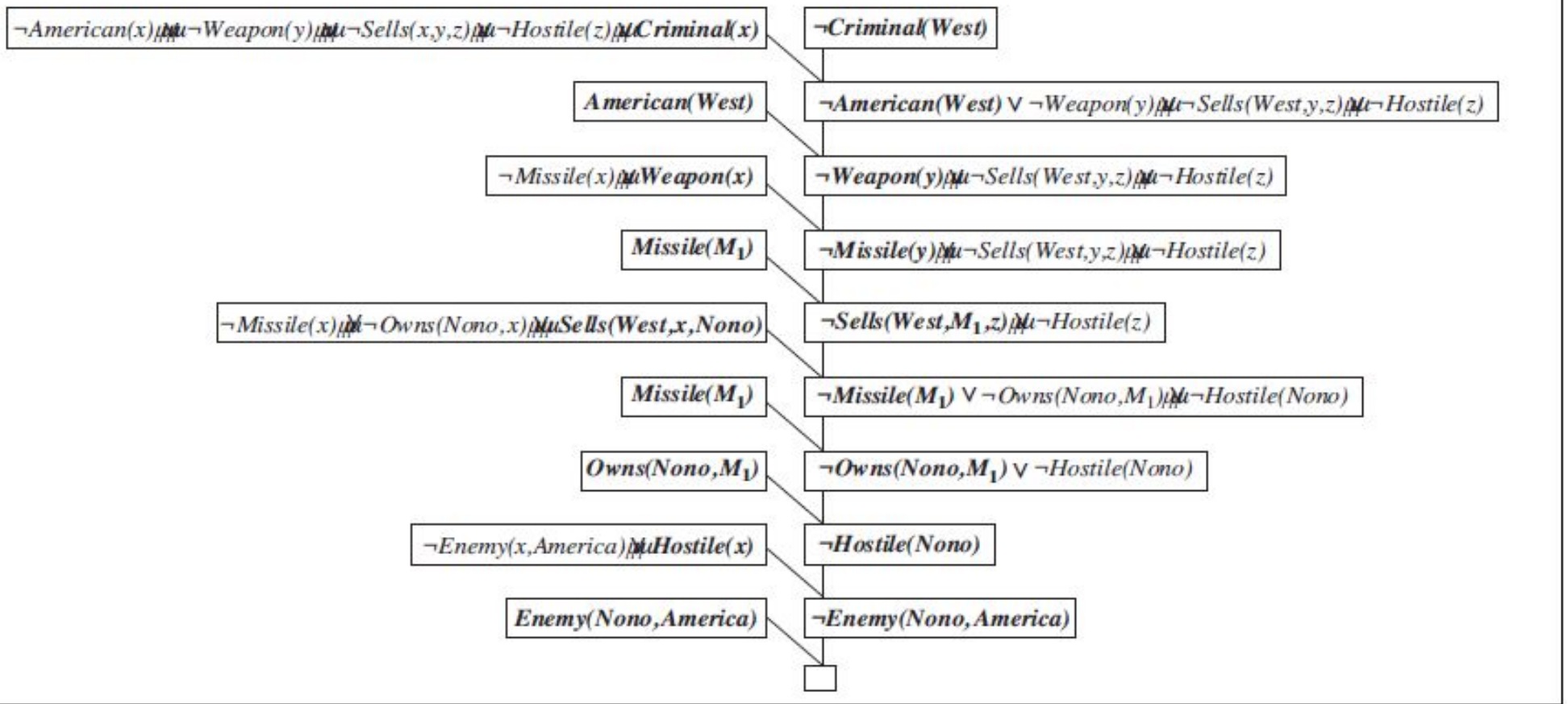


Figure 9.11 A resolution proof that West is a criminal. At each step, the literals that unify are in bold.

Step 1: Negated Goal

We want to prove:

Criminal(West)

So, we start by negating the goal:

¬Criminal(West)

This clause becomes the starting point of the resolution proof.

Step 2: Resolution Spine (Backward Chaining Style)

We now repeatedly resolve with clauses from the knowledge base (KB):

1.**¬Criminal(West)** resolves with

(¬American(x) ∨ ¬Weapon(y) ∨ ¬Sells(x,y,z) ∨ ¬Hostile(z) ∨ Criminal(x))

→ gives: ¬American(West) ∨ ¬Weapon(y) ∨ ¬Sells(West, y, z) ∨ ¬Hostile(z).

2. ¬American(West) resolves with

American(West)

→ cancels out, leaving: ¬Weapon(y) ∨ ¬Sells(West, y, z) ∨ ¬Hostile(z).

3. ¬Weapon(y) resolves with

Missile(x) ∨ Weapon(x)

→ substitution y = M1 gives: ¬Missile(M1) ∨ ¬Sells(West, M1, z) ∨ ¬Hostile(z).

4. Missile(M1) from KB resolves with ¬Missile(M1):

→ result: ¬Sells(West, M1, z) ∨ ¬Hostile(z).

5. ¬Sells(West,M1,z)resolves with

¬Missile(x) ∨ ¬Owns(Nono,x) ∨ ¬Sells(West,x,z)

→ substitution x = M1 gives: ¬Owns(Nono, M1) ∨ ¬Hostile(Nono).

6. Owns(Nono, M1) (from KB) cancels ¬Owns(Nono, M1):

→leaves ¬Hostile(Nono).

7.¬Hostile(Nono) resolves with

¬Enemy(x,America) ∨ Hostile(x)

→ gives ¬Enemy(Nono, America).

8.Enemy(Nono, America) (from KB) cancels ¬Enemy(Nono, America):

→ Empty clause reached.

Step 3: Conclusion

Since the empty clause is derived, the negated goal ¬Criminal(West) leads to contradiction.

Therefore, Criminal(West) is true.

- Our second example makes use of Skolemization and involves clauses that are not definite clauses. This results in a somewhat more complex proof structure. In English, the problem is as follows:

  Everyone who loves all animals is loved by someone.

  Anyone who kills an animal is loved by no one.

  Jack loves all animals.

  Either Jack or Curiosity killed the cat, who is named Tuna.

  Did Curiosity kill the cat?

- First, we express the original sentences, some background knowledge, and the negated goal
- G in first-order logic:

A. $\forall$ x [$\forall$ y Animal (y) $\Rightarrow$ Loves(x, y)] $\Rightarrow$ [$\exists$ y Loves(y, x)]

B. $\forall$ x [$\exists$ z Animal (z) $\wedge$ Kills(x, z)] $\Rightarrow$ [$\forall$ y ¬Loves(y, x)]

C. $\forall$ x Animal(x) $\Rightarrow$ Loves(Jack, x)

D. Kills(Jack, Tuna) $\vee$ Kills(Curiosity, Tuna)

E. Cat(Tuna)

F. $\forall$ x Cat(x) $\Rightarrow$ Animal (x)

¬G. ¬Kills(Curiosity, Tuna)

- Now we apply the conversion procedure to convert each sentence to CNF:

A1. Animal(F(x)) ∨ Loves(G(x), x)

A2. ¬Loves(x, F(x)) ∨ Loves(G(x), x)

B. ¬Loves(y, x) ∨ ¬Animal (z) ∨ ¬Kills(x, z)

C. ¬Animal(x) ∨ Loves(Jack, x)

D. Kills(Jack, Tuna) ∨ Kills(Curiosity, Tuna)

E. Cat(Tuna)

F. ¬Cat(x) ∨ Animal (x)

¬G. ¬Kills(Curiosity, Tuna)

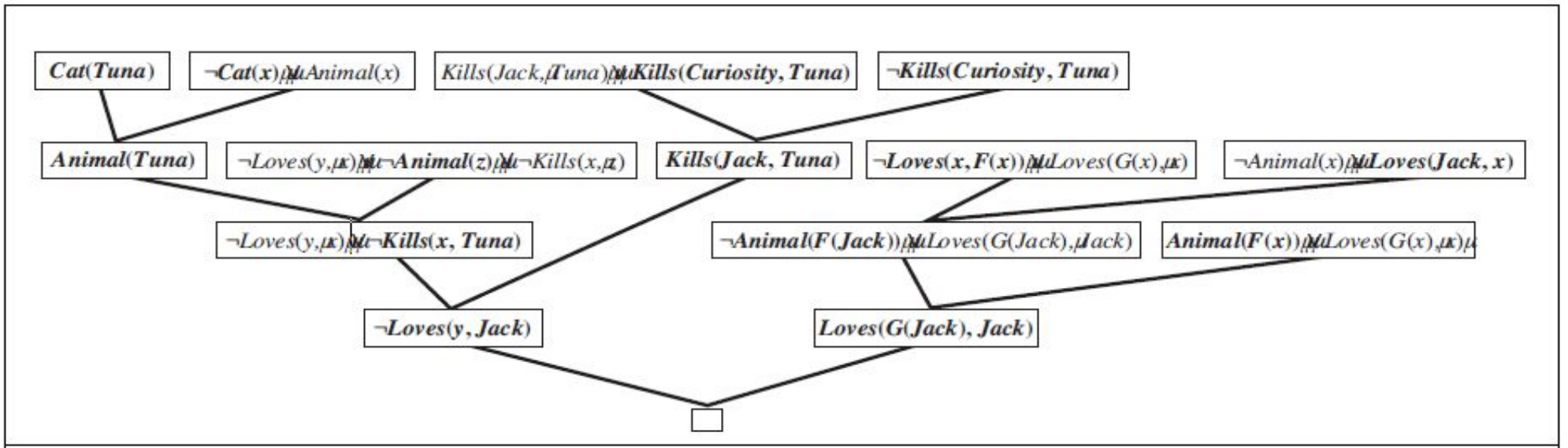


Figure 9.12 A resolution proof that Curiosity killed the cat. Notice the use of factoring in the derivation of the clause $Loves(G(Jack), Jack)$. Notice also in the upper right, the unification of $Loves(x, F(x))$ and $Loves(Jack, x)$ can only succeed after the variables have been standardized apart.

1. Facts introduced:

Cat(Tuna)

Animal(Tuna)

Kills(Jack, Tuna)

Kills(Curiosity, Tuna)

2. Rules (in clause form):

¬Cat(x) ∨ Animal(x)

¬Loves(y, x) ∨ ¬Animal(z) ∨ ¬Kills(x, z)

¬Animal(x) ∨ Loves(Jack, x)

¬Loves(x, F(x)) ∨ ¬Loves(G(x), x)

These encode statements about cats, animals, love, and killing.

3. Resolution steps:

- From Cat(Tuna) and ¬Cat(x) ∨ Animal(x) we infer Animal(Tuna).
- From Kills(Curiosity, Tuna) and rules about love/killing, we eventually derive conditions on Loves(...).
- Through unification of variables and factoring, we reduce the clauses.

4.Critical unification (upper right of the figure): Loves(x, F(x)) and Loves(Jack, x) → to unify, variables must be standardized apart (renamed).

- Example:
- Loves(x, F(x)) (with x)
- Loves(Jack,y)(with y)

  Only after renaming x → u, y → v, can we unify consistently.

5. Final derived clause:

Loves(G(Jack), Jack)

Then, combining with its negation (¬Loves(y, Jack)), we reach contradiction (empty clause).

- To answer "Who killed the cat?" we negate the query $\exists$w Kills(w, Tuna) as ¬Kills(w, Tuna). Resolution can then provide variable bindings (e.g., {w/Curiosity}). However, resolution may give nonconstructive proofs (e.g., w could be Jack or Curiosity).
- To avoid this, we can either restrict variable bindings to one per proof with backtracking, or add an answer literal (¬Kills(w, Tuna) $\vee$ Answer(w)), so the proof yields concrete answers like Answer(Curiosity).

3.9.4 Completeness of resolution

- Resolution is refutation-complete: if a set of clauses is unsatisfiable, resolution will eventually derive a contradiction. The proof uses
- (1) Herbrand's theorem to reduce to ground instances,
- (2) completeness of propositional resolution for ground sentences, and
- (3) the lifting lemma to extend ground proofs back to first-order logic.

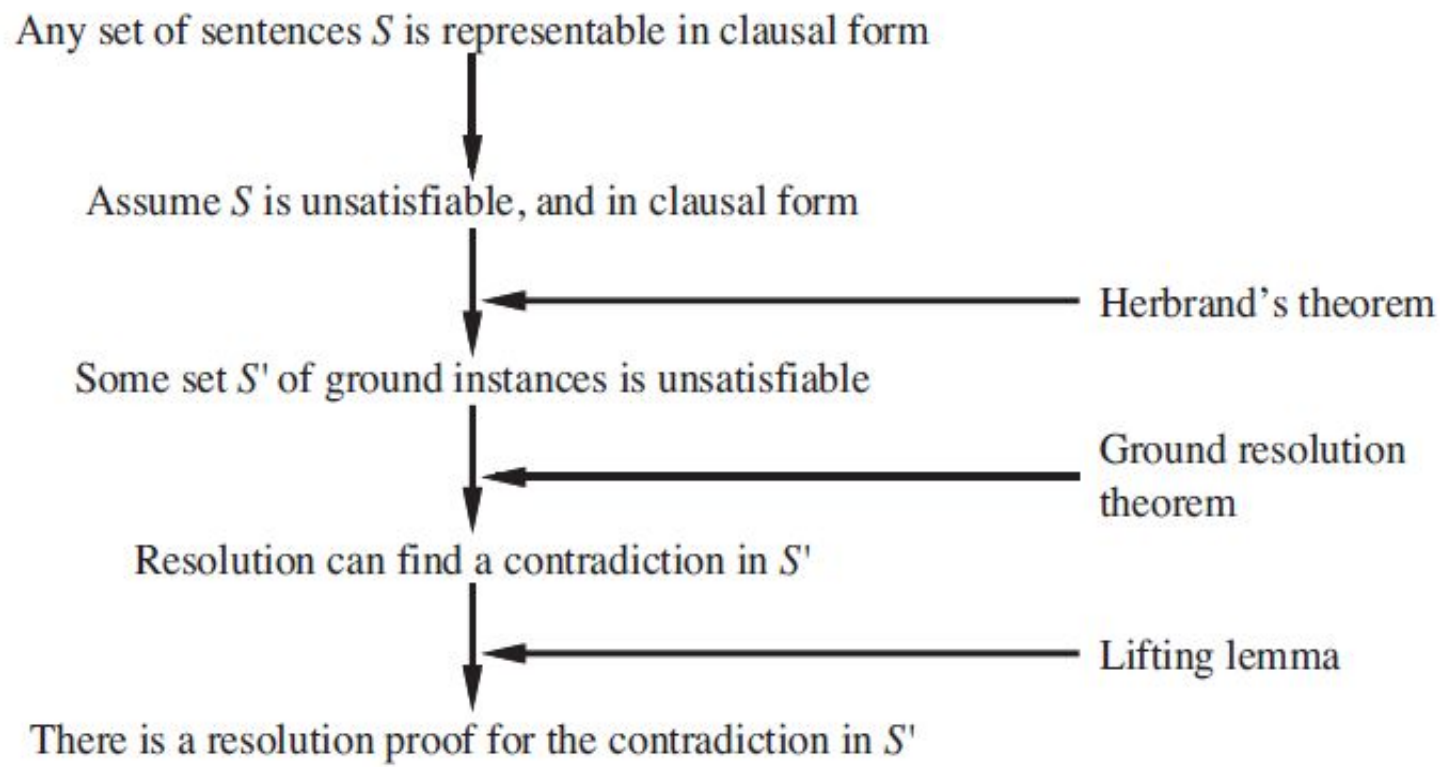


Figure 9.13 Structure of a completeness proof for resolution.

To carry out the first step, we need three new concepts:

• **Herbrand universe:** If S is a set of clauses, then H_S, the Herbrand universe of S, is the set of all ground terms constructable from the following:

a. The function symbols in S, if any.

b. The constant symbols in S, if any; if none, then the constant symbol A.

For example, if S contains just the clause $\neg P(x, F(x,A)) \vee \neg Q(x,A) \vee R(x,B)$, then

H_S is the following infinite set of ground terms:

{A,B, F(A,A), F(A,B), F(B,A), F(B,B), F(A,F(A,A)), . . .}

Saturation: If S is a set of clauses and P is a set of ground terms, then P(S), the saturation of S with respect to P, is the set of all ground clauses obtained by applying all possible consistent substitutions of ground terms in P with variables in S.

Herbrand base: The saturation of a set S of clauses with respect to its Herbrand universe is called the Herbrand base of S, written as $H_S(S)$. For example, if S contains solely the clause just given, then $H_S(S)$ is the infinite set of clauses

$\{\neg P(A, F(A,A)) \vee \neg Q(A,A) \vee R(A,B),$

$\neg P(B,F(B,A)) \vee \neg Q(B,A) \vee R(B,B),$

$\neg P(F(A,A), F(F(A,A),A)) \vee \neg Q(F(A,A),A) \vee R(F(A,A),B),$

$\neg P(F(A,B), F(F(A,B),A)) \vee \neg Q(F(A,B),A) \vee R(F(A,B),B), \ldots\}$

These definitions allow us to state a form of Herbrand's theorem

3.9.5 Equality

Axiomatization of equality in first-order logic:

Reflexive:

$$\forall x \ (x = x)$$

Symmetric:

$$\forall x \forall y \ (x = y \Rightarrow y = x)$$

Transitive:

$$\forall x \forall y \forall z \ \big((x = y \wedge y = z) \Rightarrow x = z\big)$$

Substitution for predicates (example for predicate P):

$$\forall x \forall y \ \Big(x = y \Rightarrow \big(P(x) \Leftrightarrow P(y)\big)\Big)$$

Substitution for functions (example for function f):

$$\forall x \forall y \ \big(x = y \Rightarrow f(x) = f(y)\big)$$

The first three handle equality as a relation, while the last two handle replacing equals inside predicates and functions.

1. **Axiomatization (basic method)**

- Add axioms: Reflexive, Symmetric, Transitive, Substitution.
- Works, but generates many useless conclusions → inefficient.

2. Inference rules (more efficient)

Demodulation:

- Rule: $x=y, \alpha \Rightarrow \alpha[y/x]$
- Always replace x with y (directional).
- Example:
- Father(Father(x))=PaternalGrandfather(x)
  Birthdate(Father(Father(Bella)),1926)
  Birthdate(PaternalGrandfather(Bella),1926)

Paramodulation: Extends demodulation to non-unit equalities.

- Example: From
  - $P(F(x,B),x) \vee Q(x)$
  - $F(A,y)=y \vee R(y)$
  - $P(B,A) \vee Q(A) \vee R(B)$
- Gives a complete inference procedure with equality.

Equational unification

- Extend unification algorithm itself.
- Terms unify if provably equal under axioms.
- Example: 1+2 and 2+1 unify if commutativity known.
- Efficient, specialized algorithms (commutativity, associativity, etc.).

Axioms = simple but inefficient.

Demodulation/Paramodulation = inference-rule based, complete.

Equational unification = most efficient, built into unification.

3.9.6 Resolution strategies

1. **Unit Preference / Unit Resolution**

- Prefer resolutions involving unit clauses (single literals).
- Produces shorter clauses → faster proofs.
- Unit resolution = every step must involve a unit clause.
- Complete only for Horn clauses.

2. Set of Support

- Require each resolution to involve at least one clause from a special set of support.
- Resolvent added back into set.
- If set is chosen well → efficient, goal-directed proofs.
- Complete if KB – S is satisfiable.

3. Input Resolution

- Every step uses one original input sentence (KB or query).
- Proof shape: “single spine.”
- Complete for Horn KBs, incomplete otherwise.
- Linear resolution (generalization): allow ancestor clauses too → complete.

4. Subsumption

- Eliminate clauses that are subsumed (more specific than existing ones).
- Example: if P(x) exists, drop P(A) or P(A) $\vee$ Q(B).
- Keeps KB and search space small.

Unit preference → shorter clauses.

Set of support → goal-directed, efficient.

Input/Linear resolution → smaller proof space.

Subsumption → prune redundant clauses.